UNIVERSITY OF CAMBRIDGE
Department of Computer Science and Technology

May 22, 2025

Efficient Coinductives through State-Machine Corecursors

William Sørensen

Gonville & Caius College

Submitted in partial fulfilment of the requirements for the Computer Science Tripos, Part II

Declaration

I, William Sørensen of Gonville & Caius College, being a candidate for the course, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose.
Signed: William Sørensen
Date: May 22, 2025

Abstract

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit.

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit.

Contents

1 Introduction ... 8
1.1 Use cases ... 8
1.1.1 Infinite datastructures ... 8
1.1.2 Compiler semantics ... 8
1.1.3 Safe webservers ... 8
2 Preparation ... 9
2.1 Dependent type theory ... 9
2.1.1 Categorically ... 10
2.2 Polynomial functors ... 11
2.2.1 Common pfunctors ... 11
2.2.2 Lean formalization ... 11
2.2.3 F-(co)algebras ... 12
2.3 TODO: get alex's masters ... 12
2.4 Recursion and Inductives ... 12
2.4.1 Categorically ... 12
2.5 Corecursion and coinductives ... 13
2.5.1 Morally ... 13
2.5.2 In OCaml ... 13
2.5.3 Categorically ... 13
2.6 Engineering work ... 13
2.7 Requirements ... 13
3 Implementation ... 14
3.1 Stream implementation ... 14
3.2 Expanding the progressive approximation theory ... 14
3.2.1 Universe lifting of polynomial functors. ... 14
3.2.2 Generalizing the corecursor ... 15
3.3 State machine encoding ... 15
3.3.1 PreM ... 15
3.4 Interaction trees ... 15
3.5 The ABI Type ... 15
3.5.1 Weak univalence ... 17
3.5.2 Relation to Shrink and further universe transforming types ... 17
4 Evaluation ... 18
4.1 Performance between SME and PA ... 18
5 Conclusions ... 20
Bibliography ... 21
6 Appendicies ... 22
6.A Graphical p-functors ... 23

6.A.A Common pfunctors 24

Word count: 2359

Chapter 1

Introduction

1.1 Use cases

1.1.1 Infinite datastructures

Often, a problem in programming requires leaving the finite. In cases we want to maintain safety properties of the language. For example:

- We cannot use termination as a critaria for infinite structures, so how do we maintain the totallity of the language.
- One would also want functions written for a language like this to have the expected runtime; we want a constant time to unfold.
- We want an interface that lets us unfold and reasona bout definition just how we would any other function.

A concreate example of a usecase for this would be reading from a UNIX file. Since we can for example have a program `cat` which echos out all it is fed, then what would the output of `yes | cat` be? This would have to be a stream.

1.1.2 Compiler semantics

Recalling L1[1], we gave it a big-step operational semantic. We later tried to implement an interpreter for this semantic to execute L1 programs. This required us to prove quite a few substantial coherences. An alternative way we could have tried would be to write down an interpreter, and simply use this as the semantic. Most proof-assistants would not allow this though, as they require function totallity. So we would want:

- A way to write an interpreter of a partial language, and be able to use this interpreter as a semantic in formal software.
- We would also want the asymptotics of the interpreter to be the same as the source language.
- We want to be able to encode effects in the langauge in addition to just non-termination.

1.1.3 Safe webservers

[1]TODO: Add reference to sem of prog

Chapter 2

Preparation

2.1 Dependent type theory

We can start by adding a type for Types, giving us a higher order logic. Recall back to discrete maths[2], how functions between sets $A, B : \mathbf{Type}$, $A \to B$ correspond with a form of arbitrary product, $B^A \equiv \prod_{x:A} B$. We can generalize this by changing B to be a function $B : A \to \mathbf{Type}$, then letting us write the Π-type[3] $\prod_{x:A} B(x)$. We can also do the dual construction, and get Σ-types. The inference rules for these can be seen in Figure 1. These rules should be familiar as they are the analogous to the rules for $\forall, \exists$ in the 1st order sequent calculus, their relationship can be seen in Equation 1. If you are unfamiliar with dependent types, I recommend viewing them using this relationship. For a more in depth at dependent type theory, read [1].

$$\frac{\begin{array}{c} A : \mathbf{Type} \\ B : A \to \mathbf{Type} \end{array}}{\prod_{x:A} B(x)} \Pi\text{f}$$

(a) Π-forming rule

$$\frac{\Gamma, x : A \vdash v : B(x)}{\Gamma \vdash \lambda x.v : \prod_{x:A} B(x)} \Pi\text{i}$$

(b) Π-intro rule

$$\frac{\begin{array}{c} \Gamma \vdash f : \prod_{x:A} B(x) \\ \Gamma \vdash v : A \end{array}}{\Gamma \vdash fv : B[x \ / \ v]} \Pi\text{e}$$

(c) Π-elimination rule

$$\frac{\begin{array}{c} A : \mathbf{Type} \\ B : A \to \mathbf{Type} \end{array}}{\sum_{x:A} B(x)} \Sigma\text{f}$$

(d) Σ-forming rule

$$\frac{\begin{array}{c} \Gamma \vdash a : A \\ \Gamma \vdash b : B[x \ / \ a] \end{array}}{\Gamma \vdash \langle a, b, \rangle : \sum_{x:A} B(x)} \Sigma\text{i}$$

(e) Σ-intro rule

$$\frac{\begin{array}{c} \Gamma \vdash v : \sum_{a:A} B(a) \\ \Gamma \vdash a : A, b : B[x \ / \ b] \vdash x : C \end{array}}{\Gamma \vdash \text{let } \langle a, b \rangle := v \text{ in } x : C} \Sigma\text{e}$$

(f) Σ-elimination rule[4]

Figure 1: Inference rules for Π and Σ types

$$\Pi, \Sigma \underset{\text{Categorification}^5}{\overset{\text{propositional truncation}}{\rightleftarrows}} \forall, \exists \tag{1}$$

[2]TODO: Cite DM

[3]Some call this a 'dependent product', or 'dependent function', but this sometimes confuses people so I will simply call it either a Pi, or a forall.

[4]Technically this isn't the most general eliminator for Sigma's, as C can depend on the content of the type. For symetry and simplicity I omit this.

[5]Steve Awodey calls this 'categorification' [2], as it is only one direction of Curry-Howard. I know Equation 1 is just a more verbose way of writing 'Curry-Howard', but I find it useful to be verbose here.

2.1.1 Categorically

Dependent type theory is an extention on the simply typed lambda calculus. It is for locally cartesian closed categories, what STλC is for cartesian categories; the semantics of a dependent calculus will be in terms of some LCCC.

2.2 Polynomial functors

A (multivariate) polynomial functor on set[6] is: a head-type, along with a (collection of) child family(ies) parameterised by the head. An object of a polynomial functor is a select head type, and the functions form the corresponding child(ren) parameterised by the head as seen in Equation 2[7]. Moraly polynomial functors can be thought of as types generic in some set of arguments, with a collection of constructors (the head type), where the children correspond to how many of the polymorphic argument are wanted.

$$(h : H) \times c_h \to \alpha \equiv \sum_{h:H} \alpha^{c_h} \tag{2}$$

Polynomial functors have all finite products and coproducts. Polynomial functors are also closed under (co)fixed points so I will write them using a notation a la inductives, I will use notation provided in Section 2.3. For an explanation of the notation see Section 6.A.

2.2.1 Common pfunctors

2.2.1.1 Constant

```
inductive Const
    (T : Type) (a_1 ... a_n : Type) where
  | mk (t : T)
```

(a) Inductive Notation

$$(T : \text{Type}) \to \langle T, \lambda i.\lambda h.0 \rangle \tag{3}$$

(b) Polynomial

Figure 2: Constant pfunctor

2.2.1.2 Prj

2.2.1.3 Sum

```
inductive Sum (A B : Type) where
  | inl : A → Sum A B
  | inr : B → Sum A B
```

(a) Inductive Notation

$$\langle \mathbb{B}, \lambda\{0, \text{true } 0 \mapsto 1, \text{false } 1 \mapsto 1, _ \quad _ \mapsto 0\} \rangle \tag{4}$$

(b) Polynomial

Figure 3: Constant pfunctor

2.2.1.4 Option

2.2.1.5 Functions from fixed types

2.2.1.6 W

2.2.1.7 M

2.2.2 Lean formalization

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo,

[6]For the readers familiar with category theory, simply think of categorical dependent products, and (co)limits when I use the set-theoretic language.

[7]More can be read on the ncatlab article [3], we restrict ourselves to Set, so I recommend skipping to that section.

cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

2.2.3 F-(co)algebras

2.3 TODO: get alex's masters

2.4 Recursion and Inductives

We are all familiar with algebraic datatypes from OCaml. These are structures freely generated from a set of constructurs. The classic example is a list, given as two constructors, `cons : 'a → 'a list → 'a list` and `nil : 'a list`, and a way to consume lists `fold : 'a → ('b → 'a → 'a) → 'b list → 'a`. We write this as seen in Listing 1. This is what is known as an inductive datatype. Inductive datatypes are well-founded trees[8] any function that terminates, must by definition return at most a finite (but unbounded) amount of constructors. This means fold is guarangteed to terminate too. All (primitive) recursive functions can be compiled into a call to fold[9].

```
type ('a, 'b) listF = Nil | Cons of 'a * 'b

type 'a list = Nil | Cons of 'a * 'a list
let nil = Nil
let cons h t = Cons(h,t)
let rec fold bh ih = function | Nil → bh | Cons(h, t) → ih h (fold bh ih t)
```

Listing 1: Definition of list in OCaml

2.4.1 Categorically

Given the family of endofunctors on **Set**. Consider the polynomial given by $L_A = 1 \oplus (\underline{A} \otimes \mathrm{id})$, or in its expanded form in Equation 5. Now recall the definition of the category of L_A-algebras[10]. The lists we are used to are exactly the initial object in this category. The initial map takes an algebra $f : L_A(B) \to B$ and yields a map $\mathrm{fold} : \mu L_A \to B$[11]. This is exactly what we want, as expanding the definition or $L_A(B)$ and distrubuting this over the arrows we get the signature above.

$$\begin{aligned} L_A(\quad X \quad) &= \quad 1 \quad \oplus \quad (A \quad \otimes \quad X) \\ L_A(\quad f \quad) &= (\iota_l \circ !) \oplus_m (\mathbb{1}_A \otimes_m f\ ) \end{aligned} \tag{5}$$

[8] OCaml is quite stupid and allows for non well-founded trees, an example is `let rec degen = Cons((), degen)`. This is because inductives and coinductives are 'coincident' in OCaml. We will come back to this in the next section.

[9] TODO: Cite McBride's work

[10] TODO: Cite algebra definition

[11] TODO: Cite Neel

2.5 Corecursion and coinductives

Coinductives are the duals of inductives, they have an operation `unfold` which is the dual of the operation `fold`.

2.5.1 Morally

Morally, if you have never seen `unfold`'s or coinductives, simply view it as if a fix F yields an X, then cofix F yields a lazy X[12]. We can think back to what we did in Foundations of Computer Science, when we made lazy lists we also had to change recursive occurances to be thunked, this thunking allows infinite structures to exist in finite memory.

2.5.2 In OCaml

```
type 'a colist = Nil | Cons of 'a * (() → 'a colist)
let nil = Nil
let cons h t = Cons(h,() → t)
let rec fold bh ih = function | Nil → bh | Cons(h, t) → ih h (fold bh ih t)
```

Listing 2: Definition of list in OCaml

2.5.3 Categorically

2.6 Engineering work

2.7 Requirements

[12] I. e. fix L_A gives lists, then cofix L_A gives lazy lists.

Chapter 3

Implementation

This chapter describes the implementation of the `Sme` constructs.

3.1 Stream implementation

3.2 Expanding the progressive approximation theory

During the pheasability assesment I noticed that, in the current formalised theory of polynomials, the statement wouldn't even type-check. This stemmed from a problem with the corecursive principle for the M type in the old implementation. corec : $\{\alpha : \text{TypeVec}.\{\mathcal{U}\}n\} \to \{\beta : \text{Type } \mathcal{U}\} \to (g : \beta \to P(\alpha ::: \beta)) \to \beta \to M\alpha$[13] [4]. The problem here is that both α and β have to both reside in $\mathcal{U}$. Solving this is done through the next two sections.

3.2.1 Universe lifting of polynomial functors.

The main problem caused here comes from the fact that lean isnt cummulative. This means it is impossible to express a universe hetrogouns typevector. In other words $\alpha ::: \beta$ is only typable if $\alpha : \text{TypeVec}.\{\mathcal{U}\}n$ and $\beta : \text{Type } \mathcal{U}$. The natural way of solving this is using the supremum in universe levels you get from ULift : Type $\mathcal{U} \to$ Type $(\max \mathcal{U}\mathcal{V})$. This means we can have β : Type $\mathcal{U}$ and α : Type $\mathcal{V}$, then ulift both of them to a common universe ULift $\alpha ::: $ ULift β : TypeVec.$\{\max \mathcal{U}\mathcal{V}\}(n+1)$[14].

Noticable the next hurdle we encounter is that PFunctors are restricted to a universe level. Recall the definition from Section 2.2.2. Observe how for a MvPfunctor.$\{\mathcal{U}\}n$, we require that both the head and child reside in $\mathcal{U}$. This will also cause problems, as looking back at the definition of the corecursor, we will require P to be able to accept ULift $\alpha ::: $ ULift β. If we do not add the ability to lift P, the unifier will force $\mathcal{U} = \mathcal{V}$, thereby invalidating all the work we did in the previous section. Luckily lifting a PFunctor is relatively easy. We define it as

[13] https://github.com/leanprover-community/mathlib4/blob/7a60b315c7441b56020c4948c4be7b54c222247b/Mathlib/Data/PFunctor/Multivariate/M.lean#L152-L154

[14] Note we overload ULift as a notation to refer to lifting TypeVecs as well

ULift $P \triangleq \langle \text{ULift } P.1, \lambda x \mapsto \text{ULift } (P.2x) \rangle$. This works and now we can move on to our goal[15].

3.2.2 Generalizing the corecursor

Now with all the work in the previous section, by generalizing corec'[16], we can define corecU : $\{\alpha : \text{TypeVec}.\{\mathcal{U}\}n\} \to \{\beta : \text{Type } \mathcal{V}\} \to (g : \beta \to \text{ULift } P(\text{ULift } \alpha ::: \text{ULift } \beta)) \to \beta \to M.\{\mathcal{U}\}\alpha$. Notably we are able to fit the object into $\mathcal{U}$ (as opposed to in the SME).

The expected diagram using corecU and dest commutes.

3.3 State machine encoding

Noting the definition of corecU, one might wonder if you could define M from first principles for this. The problem one encounters is one of universes. As seen in the definition above, if one were to define a type whos construcor is directly the corecU definiton, it would hold a β : Type $\mathcal{V}$. This then forces the object to reside in Type $\max \mathcal{U}(\mathcal{V}+1)$. This is a problem as one loses most closure results as you will be lifting more and more. The main beinifit from this is the performance aspect. Going from reading to a depth n, to a depth $n+1$ is not $\mathcal{O}(1)$ instead of $\mathcal{O}(n)$. This will be seen in Section 4.1. We will henceforth refer to the datatype SME.PreM.

3.3.1 PreM

As we speak about in Section 2.2.3, the M Type is the terminal object in the category of coalgebras. We can see through reasoning (cumilatively) in this category that PreM is weakly terminal. Looking at this category we want to somehow force the incoming morphisms together. This corresponds exactly to quotienting, for this we will use Bisimilarity.

3.4 Interaction trees

Interaction trees (ITrees) are a coinductive datastructure detailed in [5].

3.5 The ABI Type

The problem the ABI type tries to tackle is one of abstracting the runtime datatype through functions. Given an isomorphism eq : $\alpha \cong \beta$ for some types α and β, my first try at solving this involved constructing an object ABI $\alpha\beta$, making the following commute:

[15]TODO: Speak with JV / W to see if this might be done in the lit, Ex : Locally presentable and accessable categories Adameck roshiski

[16]Done in PR NUMBER

$$
\begin{tikzcd}
 & \beta \arrow[dd, "\mathrm{mkB}"] \arrow[ddr, "\mathbb{1}_\beta"] & \\
\alpha \arrow[ur, equal, "\mathrm{eq}"] \arrow[r, "\mathrm{mkA}"] \arrow[ddr, "\mathbb{1}_\alpha"'] & \mathrm{ABI}\ \alpha\beta \arrow[r, "\mathrm{destB}"] \arrow[dr, "\mathbb{1}_{\mathrm{ABI}\ \alpha\beta}"] \arrow[dd, "\mathrm{destA}"'] & \beta \arrow[dl, "\mathrm{mkB}"'] \arrow[ddl, equal, bend left, "\mathrm{eq}"] \\
 & & \mathrm{ABI}\ \alpha\beta \\
 & \alpha \arrow[ur, "\mathrm{mkA}"'] &
\end{tikzcd}
$$

Figure 4: Operations on ABI

Additionally I had an elimination principle satisfying the two equations below.

```
def elim : {motive : ABI _ _ eq → Type w}
        → (hLog : (z : A) → motive (mkA z))
        → (hCheap : (z : B) → motive (mkB z))
        → (eqA : ∀ z, hLog z ≍ hCheap (eq z))
        → (eqB : ∀ z, hCheap z ≍ hLog (eq.symm z))
        → (v : ABI _ _ eq) → motive v := _

theorem elimLog : {motive : carry → Type w}
        → {hLog : (z : A) → motive (mkA z)}
        → {hCheap : (z : B) → motive (mkB z)}
        → {eqA : ∀ z, hLog z ≍ hCheap (eq z)}
        → {eqB : ∀ z, hCheap z ≍ hLog (eq.symm z)}
        → ∀ z, elim hLog hCheap eqA eqB (mkA z) = (hLog z) := _
theorem elimCheap : {motive : carry → Type w}
        → {hLog : (z : A) → motive (mkA z)}
        → {hCheap : (z : B) → motive (mkB z)}
        → {eqA : ∀ z, hLog z ≍ hCheap (eq z)}
        → {eqB : ∀ z, hCheap z ≍ hLog (eq.symm z)}
        → ∀ z : B, elim hLog hCheap eqA eqB (mkB z) = (hCheap z) := _
```

Through quite a bit of work (which I call transliteration, as seen in the ABI file), You can free a universe level and open for a more general usage of the function.

3.5.1 Weak univalence

$$\begin{tikzcd}
\beta \arrow[rr, equal, "\mathrm{eq}"] \arrow[dr, "\mathrm{mkB}"'] & & \alpha \arrow[dl, "\mathrm{mkA}"] \\
& \mathrm{ABI}\ \alpha\beta &
\end{tikzcd}$$

Figure 5: Weak univalence up to shrink

3.5.2 Relation to Shrink and further universe transforming types

Chapter 4

Evaluation

4.1 Performance between SME and PA

After building the equivelence, and instantiating shrink, we now have the ability to compare the performance between 3 representations: The SME encoding (high performance, hpRuns), the PA encoding (slow, slRuns), and a hand made big implementation (bigRuns). We compare these 3 in Figure 6. In Figure 6 we see the 3 representations specalised to infinite streams, the code for which can be seen in Listing 3

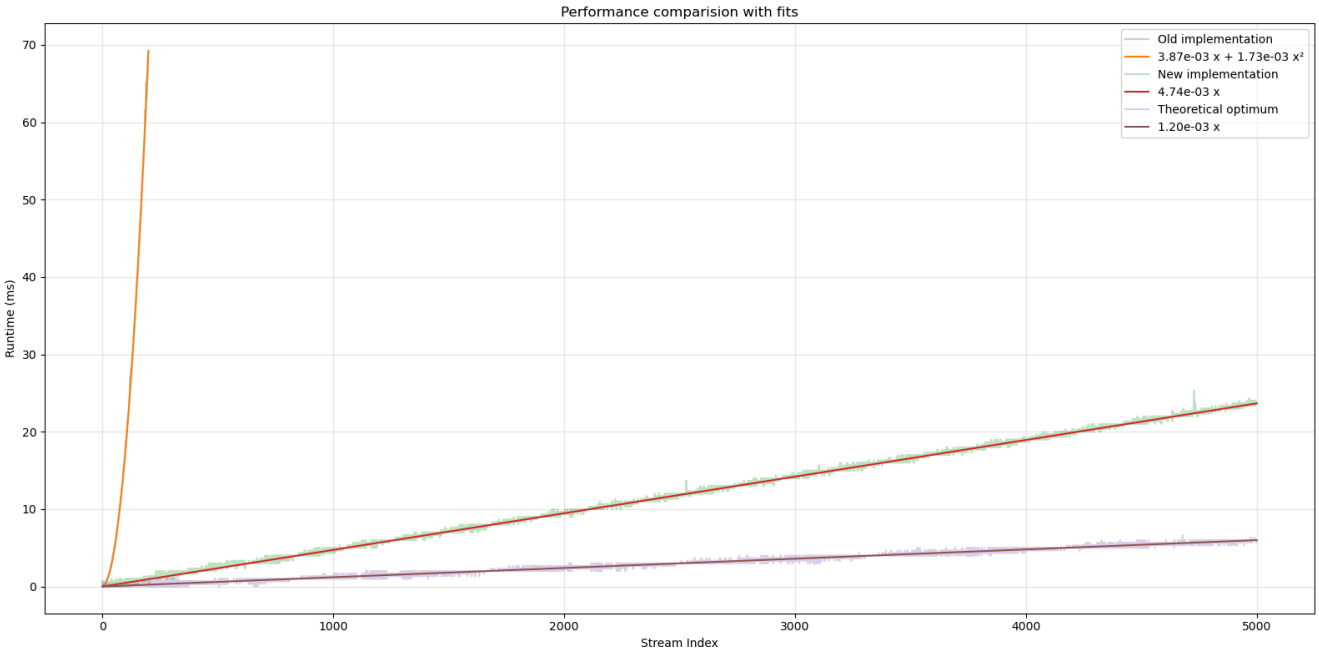

Figure 6: Performance of SME, PA, and Big representation

```
def QStream.Base : MvPFunctor 2 := ⟨
  Unit,
  fun _ _ ⇒ Unit
⟩

def QStreamSl α := M QStream.Base (fun _ ⇒ α)
def QStreamHp α := HpLuM QStream.Base (fun _ ⇒ α)

structure QStreamBig.{u} (α : Type _) where
  corec ::
    {t : Type u}
    (functor : t → Nat × t)
    (obj : t)

def numsSl : QStreamSl Nat :=
  .corec _ (fun n ⇒ ⟨.unit, fun | .fz, .unit ⇒ n.succ | .fs .fz, .unit ⇒ n⟩)
  Nat.zero

def numsHp : QStreamHp Nat :=
  .corec' (fun n ⇒ ⟨.unit, fun | .fz, .unit ⇒ n.succ | .fs .fz, .unit ⇒ n⟩)
  Nat.zero

def numsBig : QStreamBig Nat :=
  QStreamBig.corec (fun n ⇒ ⟨n, n + 1⟩) 0

def QStreamBig.getNth (x : QStreamBig Nat) : Nat → Nat
  | 0 ⇒ x.dest.fst
  | n+1 ⇒ x.dest.snd.getNth n

def QStreamSl.getNth (x : QStreamSl Nat) : Nat → Nat
  | 0 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ v (.fs .fz) .unit
  | n+1 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ QStreamSl.getNth (v .fz .unit) n

def QStreamHp.getNth (x : QStreamHp Nat) : Nat → Nat
  | 0 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ v (.fs .fz) .unit
  | n+1 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ QStreamHp.getNth (v .fz .unit) n
```

Chapter 5

Conclusions

Bibliography

[1] T. Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study: https://homotopytypetheory.org/book, 2013.

[2] Steve Awodey, "Cambridge Lectures on Categorical Logic and Type Theory." https://awodey.github.io/cltt/, Mar. 2026.

[3] nLab authors, "polynomial functor." Jan. 2026.

[4] The mathlib Community, "The Lean Mathematical Library," in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, in CPP 2020. New Orleans, LA, USA: ACM, Jan. 2020. doi: 10.1145/3372885.3373824.

[5] L.-y. Xia *et al.*, "Interaction trees: representing recursive and impure programs in Coq," *Proc. ACM Program. Lang.*, vol. 4, no. POPL, Dec. 2019, doi: 10.1145/3371119.

6 Appendicies

6.A Graphical p-functors

An alternative notation for Equation 2 can be given as: for a multivariate polynomial functor $P\alpha_0...\alpha_i$, with head types as variants $h_0...h_j$, and children c_{i,h_j}, it would conventionally be written as Equation 6, I will display the hs and cs as in Figure 7. I write an object of this functor, picked at h_0 with functions f_i, as seen in Figure 8. The justification of this notation will be seen Section 2.2.2.

$$(j' : \text{Fin } j) \times \left(h : h_{j'}\right) \times (i' : \text{Fin } i) \to c_{i',h} \to \alpha_{i'} \tag{6}$$

$$\begin{array}{c|c|c|c}
\alpha_i & c_{i,v} & ... & c_{i,v} \\
\vdots & \vdots & \iddots & \vdots \\
\alpha_0 & c_{0,v} & ... & c_{0,v} \\
\hline
 & k_0\ (v : h_0) & ... & k_j\ \left(v : h_j\right)
\end{array}$$

Figure 7: The head and child components of $P\alpha_0...\alpha_i$

$$\begin{array}{ccc}
\alpha_i & \xleftarrow{f_i} & c_{i,v} \\
\vdots & & \vdots \\
\alpha_0 & \xleftarrow{f_0} & c_{0,v}
\end{array}$$

$$\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots$$

$$k_0\ v$$

Figure 8: An object of $P\alpha_0...\alpha_i$

Mapping on MvPfunctors is done by composition of the arrows. This can be seen in Figure 9. By set forming a category, mapping a composition is the same as mapping each one by one.

$$\begin{array}{ccc}
\beta_i & \xleftarrow{g_i} & \alpha_i \\
\vdots & & \vdots \\
\beta_0 & \xleftarrow{g_0} & \alpha_0
\end{array}
\qquad
\begin{array}{ccc}
\alpha_i & \xleftarrow{f_i} & c_{i,v} \\
\vdots & & \vdots \\
\alpha_0 & \xleftarrow{f_0} & c_{0,v}
\end{array}
\qquad\qquad
\begin{array}{ccc}
\beta_i & \xleftarrow{g_i \circ f_i} & c_{i,v} \\
\vdots & & \vdots \\
\beta_0 & \xleftarrow{g_0 \circ f_0} & c_{0,v}
\end{array}$$

$$\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots

6.A.A Common pfunctors

6.A.A.A Constant

A functor, fixed at a constant type T and a arity i, denoted $\text{const}_i T$, shown in Figure 10. It has one constructor $\text{mk}_\alpha : T \to \text{const}_i T \alpha_0 ... \alpha_i$, with an inverse $\text{get}_\alpha : \text{const}_i T \alpha_0 ... \alpha_i \to T$.

α_i	0
$\vdots$	$\vdots$
α_0	0
	$\text{mk}\ (v : T)$

Figure 10: Definition of $\text{const}_i T$

$$\begin{array}{ccc}
\beta_i \xleftarrow{g_i} \alpha_i & \alpha_i \xleftarrow{¡} 0 & \beta_i \xleftarrow{¡} 0 \\
\vdots \qquad \vdots & \vdots \qquad \vdots & \vdots \qquad \vdots \\
\beta_0 \xleftarrow{g_0} \alpha_0 & \alpha_0 \xleftarrow{¡} 0 & \beta_0 \xleftarrow{¡} 0 \\
\hline
\text{<\$>} & \text{mk } v & = \quad \text{mk } v \\
\hline
 & \text{mk}_\alpha v & \text{mk}_\beta v
\end{array}$$

Figure 11: By initial hom uniqueness,

the object is invariant under mapping.

6.A.A.B Prj

Given a typevector $\alpha_0 ... \alpha_i$, and a select index into the vector $(n : \text{Fin } i)$, the projection functor holds a value of type α_n. It is defined in Figure 12. It has one constructor $\text{mk}_\alpha : \alpha_n \to \text{prj}_n \alpha_0 ... \alpha_i$, with an inverse $\text{get}_\alpha : \text{prj}_n \alpha_0 ... \alpha_i \to \alpha_n$.

$$
\begin{array}{c:c:}
\alpha_i & 0 \\
\vdots & \vdots \\
\alpha_n & 1 \\
\vdots & \vdots \\
\alpha_0 & 0 \\
\cdots\cdots & \cdots\cdots \\
 & \mathrm{mk} \\
 &
\end{array}
$$

Figure 12: Definition of prj_n

$$
\begin{array}{ccccccc}
\beta_i & \xleftarrow{g_i} & \alpha_i & \alpha_i & \xleftarrow{¡} & 0 & \beta_i \xleftarrow{¡} 0 \\
\vdots & & \vdots & \vdots & & \vdots & \vdots \\
\beta_n & \xleftarrow{g_n} & \alpha_n & \alpha_n & \xleftarrow{\boldsymbol{K}v} & 1 & \beta_n \xleftarrow{\boldsymbol{K}(g_n v)} 1 \\
\vdots & & \vdots & \vdots & & \vdots & \vdots \\
\beta_0 & \xleftarrow{g_0} & \alpha_0 & \alpha_0 & \xleftarrow{¡} & 0 & \beta_0 \xleftarrow{¡} 0 \\
\cdots & & & \cdots & & & \cdots \\
\text{<\$>} & & & \mathrm{mk} & & = & \mathrm{mk} \\
 & & & \cdots & & & \cdots \\
 & & & \mathrm{mk}_n v & & & \mathrm{mk}_n(g_n v)
\end{array}
$$

Figure 13: By initial hom uniqueness,

mapping only changes the nth argument.

6.A.A.C Sum

6.A.A.D Option

6.A.A.E Functions from fixed types

6.A.A.F W

6.A.A.G M